

Documentation technique

Vite & Gourmand

ECF - Titre professionnel Développeur Web et Web Mobile

Objectif

Réflexion technologique, configuration de l'environnement de travail, Modèle de données ou diagramme de classes, diagramme d'utilisation, diagramme de séquence et documentation de déploiement.

1. Périmètre du document

Cette documentation technique concerne l'application web Vite & Gourmand. Elle complète le README, la documentation de déploiement, le manuel d'utilisation, la documentation de gestion de projet et les documents de conception déjà présents dans le dossier docs.

L'application est un site de traiteur réalisé en PHP avec rendu côté serveur. Elle permet la consultation des menus, la commande en ligne, la gestion d'un espace client, d'un espace employé et d'un espace administrateur.

Document existant	Rôle
README.md	Installation locale, base SQL, MongoDB, e-mail, tests des rôles et upload de d'images.
docs/Déploiement_Rende_Vite_Et_Gourmand.pdf	Démarche de déploiement Render.
docs/Diagramme de classes vite.drawio.pdf	Représentation UML des principales classes et entités.
docs/Diagramme_de_cas-d'utilisation.drawio.pdf	Acteurs et cas d'utilisation du projet.
docs/Manuel d'utilisation.pdf	Parcours utilisateur pour le jury.
docs/Documentation Gestion de Projet.pdf	Organisation, planning, Git et suivi du projet.

2. Réflexion initiale technologique

Choix de l'architecture

Pour Vite & Gourmand, j'ai conservé une architecture monolithique PHP avec rendu côté serveur. Les pages PHP génèrent directement le HTML, tandis que les styles sont centralisés dans `assets/css/style.css` et que les comportements JavaScript sont regroupés dans `assets/js/app.js`.

Ce choix est cohérent avec le délai du projet, la gestion native des sessions PHP, le besoin de référencement des pages publiques et la compatibilité avec un serveur Apache. Après correction, le projet a été structuré de façon plus propre : classes Repository, classe OrderPriceCalculator, API PHP ciblées et séparation progressive de la logique front-end.

Point important

Le projet n'est pas une SPA avec un front-end totalement séparé du back-end. Il utilise un monolithe PHP structuré, avec quelques endpoints JSON pour les menus et les statistiques.

Technologies retenues

Couche	Choix technique	Justification
Front-end	HTML5, CSS3, JavaScript natif	Interface responsive, menus filtrables, interactions sans Framework JavaScript lourd.
Styles	<code>assets/css/style.css</code>	Centralisation des styles
Icônes et polices	Font Awesome et Google Fonts	Icônes de navigation et Identité visuelle du site.
Back-end	PHP 8 avec PDO	Traitement des formulaires, sessions, rôles, règles métier et Accès SQL sécurise.
SQL	MySQL/MariaDB	Stockage relationnel principal : utilisateurs, menus, plats, commandes, avis, horaires.
NoSQL	MongoDB avec fallback JSON	Statistiques administrateur. Le JSON sert uniquement de continuité si MongoDB est indisponible.
Serveur	Apache avec <code>.htaccess</code>	URL propres sans extension <code>.php</code> grâce à <code>mod_rewrite</code> .
Production	Render avec Dockerfile	PHP 8.3, Apache, <code>pdo_mysql</code> , extension MongoDB et <code>mod_rewrite</code> .

Limite de l'architecture

- L'application ne fournit pas une API REST complète pour toutes les fonctionnalités.
- Le front-end n'est pas séparé physiquement du back-end ; la separation est logique dans les fichiers.
- Les images sont stockées dans assets/images ; en production Render, un stockage persistant ou externe peut être nécessaire.
- MongoDB doit être configuré avec un vrai service distant pour répondre pleinement à l'exigence NoSQL ; le JSON est un fallback de continuité.

3. Configuration de l'environnement de travail

Environnement local

Élément	Configuration utilisée
Système local	Windows avec WampServer
Serveur local	Apache, PHP et MySQL/MariaDB fournis par WampServer
Base SQL	Base vite_et_gourmand importée depuis database.sql via phpMyAdmin
Éditeur	Visual Studio Code
Versionnement	Git et GitHub avec branches main, develop et feature
NoSQL local	MongoDB possible avec includes/mongodb_config.php ou fallback data/statistiques_menus.json

Variables de configuration

Le fichier includes/db.php lit les variables d'environnement suivantes, avec des valeurs locales par défaut adaptées à WampServer :

```
DB_HOST=localhost DB_PORT=3306 DB_NAME=vite_et_gourmand DB_USER=root
DB_PASSWORD=
```

Pour MongoDB, le projet peut utiliser le driver PHP MongoDB ou une API HTTP compatible. Les constantes et variables prévues sont documentées dans includes/mongodb_config.example.php.

```
MONGODB_URI MONGODB_DATABASE ou MONGODB_DB MONGODB_COLLECTION
MONGODB_HTTP_API_URL MONGODB_HTTP_API_KEY MONGODB_DATA_SOURCE
```

Pour les e-mails, includes/mailer.php lit les variables SMTP suivantes lorsque l'envoi réel est configuré :

```
SMTP_HOST SMTP_PORT SMTP_SECURE SMTP_USER SMTP_PASSWORD SMTP_FROM SMTP_FROM_NAME
```

Arborescence technique

Dossier ou fichier	Rôle technique
api/	Endpoint JSON : api/menus.php et api/admin_stats.php.
assets/css/style.css	Feuille de style principale de l'application.
assets/js/app.js	JavaScript natif : navigation mobile, filtres menus, graphiques, estimation commande, confirmations.
assets/images/	Images publiques et images uploadées pour les menus.
includes/	Connexion base, sécurité, mailer, statuts, historique, NoSQL.
includes/classes/	Repositories et classes métier PHP.
data/statistiques_menus.json	Fallback JSON des statistiques NoSQL.
database.sql	Script de Création et jeu de données SQL.
Dockerfile	Image de production Render avec PHP, Apache, PDO MySQL et MongoDB.

4. Architecture applicative

Flux général

Le navigateur appelle une page publique, une page protégée ou une API. Apache applique les règles .htaccess pour les URL propres, puis le fichier PHP concerne inclut les composants nécessaires : sécurité, connexion PDO, Repository ou service métier. Les pages classiques renvoient du HTML ; les API renvoient du JSON.

Etape	Description
1	Le navigateur demande une URL propre comme /menus ou /espace_admin.
2	Apache et .htaccess réécrivent l'URL vers le fichier PHP correspondant si le fichier existe.
3	Le fichier PHP inclut includes/security.php et/ou includes/db.php selon le besoin.
4	Les données sont récupérées avec PDO, souvent via une classe Repository.
5	La réponse est du HTML pour les pages ou du JSON pour api/menus.php et api/admin_stats.php.

Pages et routes principales

Type	Pages ou fichiers
Public	index, menus, menu_détail, contact, login, register, cgv, mentions, confidentialite.
Client	commande, espace_utilisateur, logout.
Employé	espace_employé.
Administrateur	espace_admin et api/admin_stats.php.
API	api/menus.php pour le catalogue, api/admin_stats.php pour les statistiques.

Composants PHP principaux

Composant	Responsabilité
includes/db.php	Connexion PDO a MySQL/MariaDB avec charset utf8mb4.
includes/security.php	Headers de sécurité, session sécurisée, CSRF, rôles, validation simple, limitation des tentatives de connexion.
MenuRepository	Catalogue, détail menu, stock, images, relations menu-plat.
OrderRepository	Création, modification, annulation, statut, avis et historique des commandes.
OrderPriceCalculator	Calcul serveur du prix : menu, remise de groupe, livraison hors Bordeaux.
UserRepository	Utilisateur, client, employé, mot de passe, activation/désactivation.
DishRepository / AllergenRepository	CRUD des plats et Allergènes.
ReviewRepository	Modération et affichage des avis.
ScheduleRepository	Gestion des horaires affichés dans le footer.
PasswordResetRepository	Demandés de réinitialisation de mot de passe.
includes/nosql_stats.php	Lecture/synchronisation des statistiques MongoDB ou fallback JSON.
includes/mailer.php	Envoi SMTP ou mail(), avec journal email_log.

Composant Javascript principaux

Classe JS	Rôle
MobileNavigation	Ouverture et fermeture du menu mobile.
MenuFilter	Chargement des menus depuis api/menus.php et filtrage côté navigateur.
AdminStatsChart	Chargement des statistiques depuis api/admin_stats.php et affichage graphique.
OrderSummaryCalculator	Estimation visuelle du total dans le formulaire de commande.
RowVisibilityController	Affichage conditionnel de champs et sections.
ConfirmSubmitController / ConfirmClickController	Confirmations avant certaines actions sensibles.

5. Modèle de donnée relationnel

La base relationnelle est créée par database.sql. Les tables sont en InnoDB avec charset utf8mb4. Les relations principales sont matérialisées par des clés étrangères.

Table	Rôle	Relations principales
utilisateur	Comptes client, employé et administrateur.	Liée à commande et avis.
menu	Menus proposés par le traiteur.	Liée à commande, menu_plat et menu_image.
plat	Plats disponibles pour composer les menus.	Liée à menu_plat et plat_allergene.
allergene	Allergènes associés aux plats.	Liée à plat_allergene.
menu_plat	Table de liaison entre menus et plats.	FK vers menu et plat, suppression en cascade.
plat_allergene	Table de liaison entre plats et Allergènes.	FK vers plat et allergène, suppression en cascade.
menu_image	Galerie et image principale synchronisées en base.	FK vers menu, suppression en cascade.
commande	Commandes client avec date, heure, lieu, personnes, total et statut.	FK vers utilisateur et menu.
commande_statut_historique	Historique des changements de statut.	FK vers commande, suppression en cascade.

avis	Avis déposés par les clients puis modérés.	FK vers utilisateur et commande.
horaire	Horaires affichés sur le site.	Indépendante.
password_reset	Jetons de réinitialisation de mot de passe.	Liée fonctionnellement a utilisateur par e-mail.
login_attempt	Limitation des échecs de connexion par e-mail et IP.	Clé unique email + ip_address.
email_log	Journal des e-mails envoyés ou en erreur.	Indépendante.

Données NoSQL

La partie NoSQL concerne les statistiques administrateur. includes/nosql_stats.php construit des documents par menu avec l'identifiant du menu, son nom, le nombre de commandes, le chiffre d'affaires et le détail des commandes. Lorsque MongoDB est disponible, les données sont lues ou synchronisées avec MongoDB. Sinon, le fichier data/statistiques_menus.json assure une continuité d'affichage.

```
{id_menu: 1, nom_menu: "Menu ...", nombre_commandes: 0, chiffre_affaires: 0, commandes: [{ date: "YYYY-MM-DD", montant: 0.00}]}
```

6. Diagrammes

Tous les diagrammes sont fournis dans docs/

Diagramme de classe

Il représente les principales classes PHP et les entités manipulées par l'application, notamment les Repository, OrderPriceCalculator et les entités liées aux utilisateurs, menus, plats, commandes, avis et statistiques.

Diagramme de cas d'utilisation

Il montre les actions principales des acteurs : visiteur, client, employé et administrateur.

Diagramme de séquence – Création d'une commande

Il représente le parcours complet de création d'une commande dans l'application. Il traverse plusieurs parties importantes du projet : le navigateur du client, la page commande.php, la vérification de la session et du jeton CSRF, les classes Repository, le calcul du prix côté serveur avec OrderPriceCalculator, la base de données SQL, l'historique de commande, la synchronisation des statistiques avec MongoDB ou le fallback JSON, puis l'envoi ou la journalisation de l'e-mail de confirmation.

Ce diagramme montre donc la logique réelle du projet : le client soumet une commande, le serveur vérifie les données, ouvre une transaction SQL, verrouille le menu concerné, contrôle le stock, recalcule le prix final, enregistre la commande, ajoute l'historique, synchronise les statistiques et redirige ensuite le client vers son espace utilisateur.

Ordre	Acteur / composant	Action
1	Client	Ouvre commande?id_menu=... après connexion.
2	commande.php	Vérifie la session, récupère le menu avec MenuRepository et l'utilisateur avec UserRepository.
3	Client	Soumet le formulaire de commande.
4	security.php	Vérifie le jeton CSRF.
5	commande.php	Valide la date, l'heure, l'adresse, le nombre de personnes et la distance hors Bordeaux.
6	PDO / MySQL	Démarre une transaction.
7	MenuRepository	Recharge le menu avec FOR UPDATE, Vérifie le stock et le minimum de personnes.
8	OrderPriceCalculator	Recalcule le total côté serveur.
9	MenuRepository	Décrémente le stock si le menu est disponible.
10	OrderRepository	Insère la commande avec le statut en_attente.
11	order_history.php	Ajoute une ligne dans commande_statut_historique.
12	PDO / MySQL	Valide la transaction avec commit.
13	nosql_stats.php	Synchronise les statistiques depuis SQL vers MongoDB ou le JSON de continuité.
14	mailer.php	Envoie ou journalise l'e-mail de confirmation.
15	commande.php	Redirige vers espace_utilisateur?success=commande_validee.

Scenarios alternatifs : si le CSRF est invalide, le serveur renvoie une erreur 403 ; si le stock ou le minimum de personnes est invalide, la transaction est annulée ; si l'e-mail ne part pas, l'application continue et journalise l'information

7. Sécurité technique mise en place

Mecanisme	Implémentation dans le projet
Sessions	secure_session_start configuré les cookies avec HttpOnly, SameSite=Lax et Secure lorsque HTTPS est actif.
Rôles	require_role et les controles de session protegent les espaces client, employé et administrateur.
CSRF	csrf_field génère un jeton et verify_csrf le Vérifie sur les POST sensibles.
SQL	PDO avec requêtes préparées et ATTR_EMULATE_PREPARES=false.
XSS	Échappement des sorties avec htmlspecialchars sur les données affichées.
Mots de passe	password_hash et password_verify sont utilisés pour l'inscription, la connexion et la réinitialisation.
Force brute	login_attempt bloque temporairement après plusieurs échecs par e-mail et IP.
Headers HTTP	CSP, X-Frame-Options, nosniff, Referrer-Policy, Permissions-Policy, HSTS sous HTTPS.
Upload images	Contrôle taille 5 Mo, MIME réel, extensions JPG/PNG/WEBP/AVIF, nom aléatoire, basename et suppression contrôlée.
URL propres	.htaccess reecrit les URL sans .php ; ce point améliore la lisibilité mais ne remplace pas les controles serveur.

8. Documentation du déploiement

Déploiement local

Le déploiement local est décrit dans README.md. Le principe est de placer le projet dans C:\wamp64\www\vite-et-gourmand, d'activer le module Apache rewrite_module, d'importer database.sql dans une base vite_et_gourmand, puis de lancer l'application depuis WampServer.

Déploiement Render

La version finale vise Render. Le Dockerfile du projet part de PHP :8.3-apache, installe pdo_mysql, l'extension MongoDB, active mod_rewrite, copie les fichiers du projet et adapte Apache au port fourni par Render.

```
FROM php :8.3-apache # Installation pdo_mysql, extension mongodb,
mod_rewrite # Exposition du port Render via la variable PORT
```

En production, les valeurs sensibles ne doivent pas être écrites dans le code. Elles doivent être configurées dans les variables d'environnement Render : Accès SQL, MongoDB et SMTP.

Élément	A configurer en production
Base SQL	DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD.
MongoDB	MONGODB_URI, MONGODB_DATABASE, MONGODB_COLLECTION ou variables API HTTP.
E-mail	SMTP_HOST, SMTP_PORT, SMTP_SECURE, SMTP_USER, SMTP_PASSWORD, SMTP_FROM.
HTTPS	Gere par l'hébergeur Render.
Images	Prévoir un stockage persistant ou externe si l'application doit conserver durablement les upload.

Le document détaillé de déploiement est fourni dans docs.

9. Tests techniques à réaliser après installation

- Ouvrir les pages publiques : accueil, menus, détail menu, contact, connexion et inscription.
- Tester la connexion et la déconnexion d'un client, d'un employé et d'un administrateur.
- Créer une commande et vérifier le recalcul serveur du prix, le stock, l'historique et l'espace client.
- Ajouter, modifier et supprimer un menu avec image principale et galerie depuis l'espace employé.
- Valider ou refuser un avis depuis l'espace employé et vérifier l'affichage public.
- Consulter api/admin_stats.php depuis une session admin et vérifier le refus d'Accès hors session admin.
- Vérifier que les URL propres fonctionnent : /menus, /contact, /login, /espace_admin.
- Contrôler que MongoDB est utilisé si configurer, sinon que le fallback JSON évite une erreur bloquante.

10. Conclusion

L'application Vite & Gourmand repose sur une architecture PHP monolithique structurée. Elle combine un rendu serveur, une base relationnelle MySQL/MariaDB, une partie NoSQL pour les statistiques, des API JSON ciblées, des classes orientées objet et une séparation plus nette des styles et du JavaScript.